Mindanao State University - Iligan Institute of Technology
College of Computer Studies
Department of Computer Applications
BCA182 Embedded Systems Programming

Last Name: _____ Date: _____
First Name: _____ Subject/Section: _____

Project 2
Logging, Monitoring, and Controlling Using the STM32F407-based Development Board
100 Points

Overview

In this project, you will design and develop your own integrated Internet of Things (IoT) monitoring and control system using the RT-Spark (Spark-1) STM32F407 Development Board. You will use its fitted onboard sensors and modules to collect measurements, record timestamped data, monitor current and historical conditions, and remotely control local outputs.

Your application must perform three connected functions: logging sensor-generated measurements in a time-series database, monitoring the measurements through a usable dashboard, and controlling at least one onboard output through an MQTT command. Select an original and technically justified application scenario, define meaningful operating conditions, and implement a working end-to-end system using the resources provided.

As 4th-year BS in Computer Applications students specializing in embedded systems, you should already have experience reading sensors, developing firmware, communicating over a network, and organizing software projects. The challenge is to integrate these skills using STM32Cube HAL/LL firmware, FreeRTOS for concurrent execution, PlatformIO for development, and GitHub for version control.

This project involves a range of independent tasks across multiple platforms, including:

- Identifying, configuring, and integrating the available onboard sensors, inputs, indicators, and applicable storage and communication interfaces
- Developing modular STM32 firmware with FreeRTOS tasks, timing, synchronization, and inter-task communication
- Publishing sensor telemetry and receiving validated remote-control commands through MQTT
- Logging real measurements in InfluxDB and producing understandable dashboards in Grafana
- Conducting repeatable tests for sensor behavior, end-to-end performance, command response, and connection recovery

Requirements

For this project, you will use the following resources:

- One RT-Spark (Spark-1, STM32F407ZGT6) Development Board with its fitted onboard sensors and modules
- A computer, USB/programming cables, an ST-LINK or other compatible debug/upload interface, and a functioning network connection

- An MQTT broker, an InfluxDB instance, and Grafana; local or cloud deployments are acceptable where supported
- Visual Studio Code with PlatformIO; the ST STM32 platform and STM32Cube framework (HAL/LL); a correctly configured FreeRTOS kernel and the drivers/middleware required by the application
- A GitHub account and Git repository for PlatformIO project files, firmware, service code, setup instructions, commits, and verification evidence

Use the actual RT-Spark board as the target hardware. Additional external sensors or actuators are optional, not substitutes for the required onboard sensors. Before selecting libraries or drivers, verify the board revision, microcontroller, pin assignments, power requirements, and communication buses. Arduino-based firmware is prohibited.

Tasks

The project involves four key tasks:

1. System design and onboard hardware inventory
2. STM32Cube firmware, FreeRTOS, and onboard peripheral integration
3. MQTT communication, InfluxDB logging, and Grafana monitoring
4. Remote control, system verification, and GitHub documentation

Begin by checking the actual board, identifying the sensors and other fitted modules, and planning the data and control flow. Establish the PlatformIO/STM32Cube/FreeRTOS build before integrating every peripheral. Test individual functions first, then demonstrate them together in one reproducible system.

High Level Overview

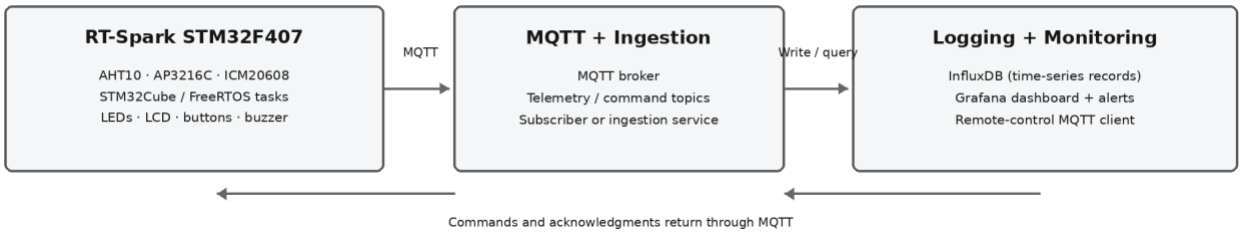


Figure 1. Proposed end-to-end architecture. The ingestion service subscribes to MQTT telemetry and writes to InfluxDB; command topics return messages to the RT-Spark board.

This figure illustrates the required data and control flow. The STM32 firmware should acquire measurements under FreeRTOS, publish selected values through MQTT, and respond to valid incoming commands. An ingestion service writes measurements to InfluxDB; Grafana visualizes the stored data. The MQTT broker is not the time-series database, and Grafana is not the actuator-control transport.

System Design and Hardware Inventory

Propose an original application and explain what will be sensed, why each measurement is meaningful, what events require local feedback, and what should be controlled remotely. Suitable domains include workspace-condition monitoring, laboratory environmental monitoring, or equipment movement and

orientation monitoring. The application must incorporate environmental, light/proximity, and motion measurements rather than use only one sensor.

Inspect the exact board assigned to your group. Prepare a complete onboard hardware inventory listing every fitted sensor, user input, indicator, communication interface, and available storage resource. For each component, identify its peripheral bus or GPIO, its proposed use, and your integration evidence. All fitted onboard sensors must be read and demonstrated; use the relevant fitted modules for local feedback, control, communication, and storage where technically applicable. Explain and substantiate any hardware or driver limitation.

Component / module	Expected project use	Evidence or limitation
AHT10 temperature/humidity sensor	Acquire onboard environmental temperature and humidity using an STM32-compatible sensor driver.	Show changing valid readings with units and timestamps.
AP3216C light/proximity sensor	Acquire light and proximity data through the appropriate onboard interface.	Demonstrate repeatable light and near-object changes.
ICM20608 accelerometer/gyroscope	Acquire accelerometer and gyroscope axes; derive a meaningful motion or orientation event.	Show raw/derived readings and a controlled motion event.
Buttons and local indicators	Use fitted buttons for local input and fitted LEDs, buzzer, and/or other indicators for safe control and status.	Demonstrate local input and observable state changes, including an MQTT command.
LCD, LED matrix, RGB/individual LEDs, IR receiver	Inventory and integrate fitted display, LED-matrix, RGB/IR or related modules where usable and relevant to the scenario.	Demonstrate fitted modules or document tested technical limitations.
Wi-Fi, RTC, SPI Flash, microSD interface	Use onboard Wi-Fi for MQTT; inspect RTC, SPI Flash, and microSD interface for timestamps, settings, or optional buffering.	Verify Wi-Fi data transfer; show local storage/time functions where media and drivers are usable.

The Spark-1 hardware documentation identifies the AHT10, AP3216C, and ICM20608 sensors, plus indicators, pushbuttons, a display and other peripheral modules. Different functions may require I2C, SPI, UART, GPIO, or other STM32 interfaces. A driver supplied for a different software environment must not be assumed compatible with STM32Cube. Adapt or implement a suitable HAL/LL driver, verify the actual pin map, and document any unavailable component or accessory.

STM32Cube and FreeRTOS Device Firmware

Your final firmware must build, upload, and execute on the actual STM32F407 board through PlatformIO. Configure the ST STM32 platform with framework = stm32cube and integrate FreeRTOS, including the necessary kernel sources, portable layer, and FreeRTOSConfig.h. Confirm the correct MCU/board manifest, clock setup, memory settings, linker/startup files, I/O pin map, and upload/debug configuration. If no exact PlatformIO board definition exists, provide and document a validated custom board configuration. STM32CubeMX may assist with peripheral setup, but final project building and programming must use PlatformIO. Do not use the Arduino framework or substitute another RTOS.

Your application must contain clearly separated responsibilities implemented as FreeRTOS tasks or well-defined supporting modules:

- Sensor-acquisition task: initialize the fitted sensors, acquire readings at justified intervals, check return status, and attach timestamps or monotonic sample counters.
- Communication task: initialize the supported onboard Wi-Fi interface and its STM32-compatible driver, maintain the broker connection, publish telemetry, subscribe to commands, and reconnect without permanently blocking sensing.
- Control and local-feedback task: validate received commands, drive a fitted LED, buzzer, display, or other safe output, respond to local button input, and report the actual device state.
- Data-transfer and synchronization mechanism: use FreeRTOS queues, semaphores, mutexes, event groups, or task notifications as appropriate to move readings and commands safely between tasks. Explain task priorities, stack allocation, shared resources, and the reason for each mechanism.

As a starting point, sample environmental and proximity measurements every 1 to 5 seconds. Motion readings may require a shorter interval, depending on the application. Publish aggregated MQTT telemetry every 5 to 10 seconds, unless measurements justify a different interval. Prefer FreeRTOS timing mechanisms such as vTaskDelayUntil over busy waiting. Document actual scheduling periods and test their behavior.

Firmware must check peripheral initialization and sensor read errors, avoid blocking unrelated tasks during network faults, and recover from Wi-Fi/MQTT disconnections. Invalid or unauthorized commands must be rejected or safely ignored. Include serial diagnostics, a defined startup/default output state, and evidence that sensing and control remain responsive under concurrent operation.

MQTT Communication

Use MQTT to transfer telemetry and receive remote control messages. You may use a managed broker or a self-hosted broker such as Mosquitto. Avoid sharing a public, unauthenticated broker for any sensitive or consequential control function. Use unique topic names and device identifiers to prevent collisions between student groups.

A suggested topic structure is shown below. You may design your own equivalent structure, but it must be documented and consistently implemented:

Direction	Example topic	Expected message
Publish	bca182/group-id/telemetry	device_id, timestamp, temperature_c, humidity_pct, light, proximity, accel, gyro
Publish	bca182/group-id/status	online/offline, current output state, selected fault/status codes
Subscribe	bca182/group-id/cmd/indicator	validated instruction: ON, OFF, or a documented mode
Publish	bca182/group-id/ack	command_id, accepted/rejected, actual resulting state, timestamp

Choose a consistent structured format such as JSON. Clearly distinguish telemetry, command, acknowledgment, and status messages. Validate received payloads and maintain the actual output state separately from a requested state. Do not commit Wi-Fi passwords, API tokens, or broker credentials to your public repository.

Time-Series Logging and Monitoring

Log real, timestamped measurements in InfluxDB. An MQTT subscriber or ingestion service may run on a computer, server, or cloud platform; it should validate payloads, assign reliable timestamps, and write

consistent measurements, fields, units, and tags. Running an InfluxDB client directly on the STM32 is not required. Document how the ingestion program is started and how network failures are handled.

Your Grafana dashboard should allow users to understand both current and historical conditions without interpreting raw MQTT messages. At a minimum, create the following views:

- Temperature and humidity over time; light/proximity measurements over time
- Movement or orientation metric over time, together with meaningful detected events
- Current device connectivity, last received data time, and commanded versus actual output state
- A selectable time range and at least one meaningful threshold/event indicator or alert

Use readable titles, units, legends, and time axes. Demonstrate a change in each applicable sensor during data collection. The report should identify the measured signal, its sampling interval, how its timestamp was generated, and the corresponding InfluxDB field. Do not present fabricated or manually entered samples as live readings.

Remote Monitoring and Control

Provide an interface through which an authorized user can issue at least two distinct MQTT control commands to a fitted onboard output, such as a selectable LED mode, buzzer alert, or display status. The interface may be an MQTT desktop/mobile client or a simple web interface. Grafana is used for visualization; a separate MQTT publisher is acceptable and often preferable for actuation.

At minimum, demonstrate (1) a valid remote command, (2) a visible or audible change in the selected local output, (3) an acknowledgment that reflects the actual result, and (4) a local input that changes the mode or state according to documented priority rules. A servo motor is not required unless one has been separately provided and approved.

Verification and Evaluation

Prepare a test matrix before your final demonstration. For each case, list the input or test condition, expected behavior, observed behavior, and pass/fail result. Confirm the actual firmware was built in PlatformIO and is running with FreeRTOS tasks on the assigned STM32 board.

Test category	Minimum demonstration
Onboard peripherals	Onboard AHT10, AP3216C, and ICM20608 show controlled sensor changes; fitted input/output and feedback modules are checked.
FreeRTOS task behavior	Sensing, MQTT handling, and local control execute concurrently using FreeRTOS; demonstrate task timing and a queue, semaphore, mutex, or task notification.
Telemetry and persistence	A device-generated MQTT message is stored with a plausible timestamp and appears consistently in InfluxDB and Grafana.
Remote command	Two valid commands change a local output; malformed commands are rejected; acknowledgment reflects the actual state.
Failure and recovery	Disconnect network/broker temporarily; sensor tasks remain responsive where feasible; MQTT operation resumes without reflashing.
Sustained run	Run continuously for at least 30 minutes, record expected vs. received telemetry, and report errors, resets, and task health.

For remote control, perform at least 10 command-response trials and report the minimum, median, and maximum response times. During a continuous 30-minute run, compare expected and received telemetry messages, record missing data and disconnections, and identify any task, timing, or sensor faults. Support reliability claims with observed results rather than a single successful demonstration.

Project Deliverables

Submit the following materials through the channel and deadline announced by the instructor:

1. A working RT-Spark STM32F407 prototype running STM32Cube-based firmware with FreeRTOS and demonstrating all required project functions on the actual board.
2. A GitHub repository containing platformio.ini, any custom board configuration, STM32Cube HAL/LL and FreeRTOS project configuration, firmware sources, ingestion service, README, test artifacts, and meaningful Git commit history. Exclude credentials and generated temporary files.
3. A laboratory report describing the application problem, complete system architecture, board/component inventory, FreeRTOS task design and synchronization, MQTT topics/payloads, InfluxDB schema, Grafana panels, and local/remote control logic.
4. Photographs and screenshots of the actual board and onboard modules, PlatformIO build/upload output, firmware diagnostics, MQTT telemetry and commands, database records, Grafana dashboard, and actual command acknowledgments.
5. A verification matrix showing sensor and module tests, task operation, safe command handling, the 30-minute continuity test, at least 10 response-time trials, observed limitations, and justified exclusions.

Your submission must allow another person to rebuild, upload, and demonstrate the system by following the GitHub README without undocumented instructions. Any external library, sample driver, or borrowed code must be identified and properly acknowledged. Working integration and demonstrated understanding remain mandatory.

Assessment Criteria (100 Points)

Criterion	Assessment basis	Points
System concept and architecture	Original, coherent application; accurate diagram; complete hardware inventory and traceable requirements	10
Onboard peripheral integration	Correct sensing and fitted module integration, inputs/outputs, feedback and documented technical limitations	20
STM32Cube/FreeRTOS firmware design	PlatformIO firmware; concurrent FreeRTOS tasks, priorities, synchronization, modularity, and error handling	20
MQTT and remote control	Telemetry schema; subscribed commands; validation, acknowledgment and actual state control	15
InfluxDB and Grafana	Consistent timestamped storage, useful dashboard panels, units, time ranges, and alerts	15
Testing and reliability	Repeatable functional and end-to-end tests, network recovery, sustained run, and command-response measurements	10
Documentation and repository	GitHub repository with PlatformIO config, build steps, task design, evidence, meaningful commits and secure credentials	10
TOTAL		100

A complete system must demonstrate all three capabilities in the project title: logging, monitoring, and controlling. A dashboard without sensor-originated data, a command widget without working onboard actuation, or firmware not built using the required STM32Cube, PlatformIO, and FreeRTOS stack does not constitute a complete submission.

Implementation Rules

Required firmware stack: STM32Cube HAL/LL, FreeRTOS, and PlatformIO on the RT-Spark STM32F407 board. Submit the build configuration and show a successful device upload and execution. The Arduino framework is not permitted.

PlatformIO does not guarantee a ready-made configuration for every board or automatically supply a complete FreeRTOS project. Verify and include the correct target configuration, RTOS port, low-level drivers, middleware, and any adapted peripheral code.

GitHub documentation must identify all dependencies, peripheral mappings, test procedures, and limitations. Do not publish Wi-Fi passwords, MQTT credentials, InfluxDB tokens, or other secrets. Use documented placeholders or protected local configuration.